

Transaction Processing Pipeline

A four-stage payment pipeline built end to end by four AI workflow agents — specification, code, tests, documentation — where every decision stays explainable after the fact.

Andrii Shukailo · @overgapo

THE PROBLEM

Money code fails quietly

The defects that matter here look, in a diff, exactly like ordinary working code.

- A **float** where a `Decimal` belonged — off by a cent, found at reconciliation weeks later.
- An **account number in a log line** — invisible until an audit.
- A **filing treated as a hold** — a compliance obligation quietly turned into a blocked payment.
- A **guessed FX rate** when the real one was missing — plausible, wrong, and unreviewable.

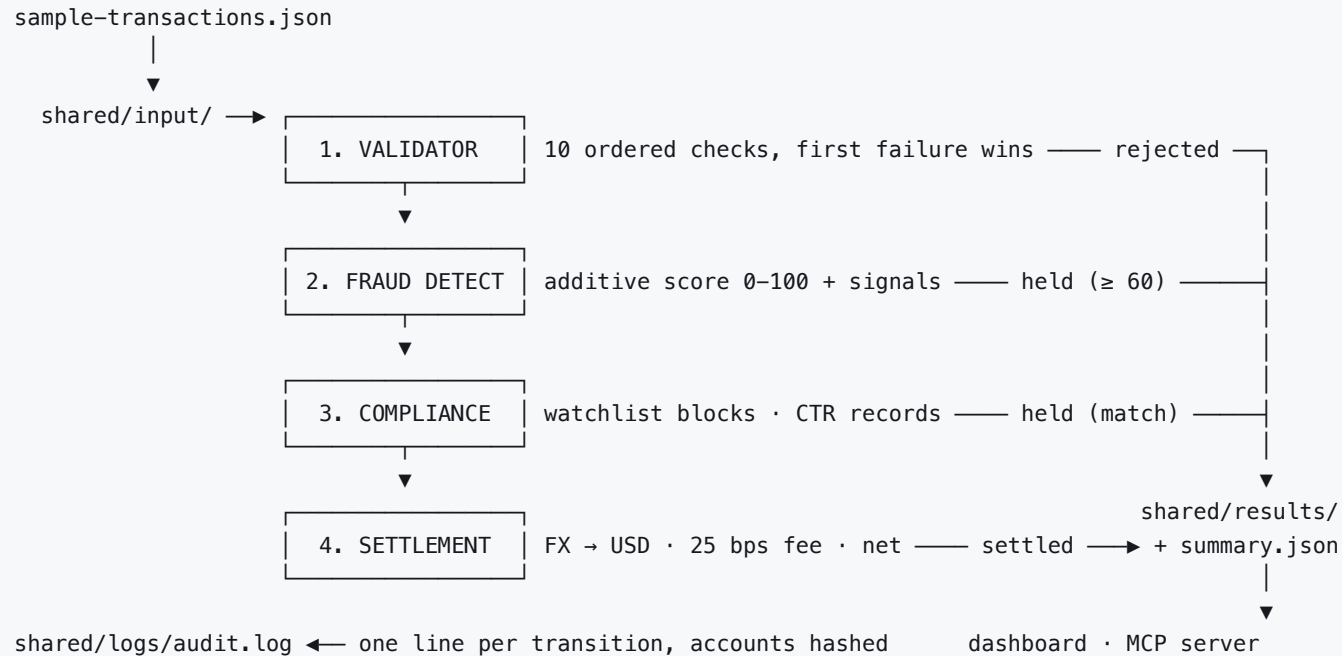
So the design goal was not "make it work". It was: make every outcome reconstructable by hand, months later.

What the pipeline answers, per transaction

Not just *what* happened — *why*, and *who* decided.

```
"final_status": "held",
"reason_code": "HIGH_RISK",
"decided_by": "fraud_detector",
"risk": {
  "score": 60,
  "band": "HIGH",
  "signals": ["HIGH_VALUE",
              "VERY_HIGH_VALUE"]
},
"fx": { "rate": "1.0850",
        "version": "2026-03-16.1" }
```

Four stages, talking only through files



A stage claims a record into `processing/`, writes atomically (`.tmp + os.replace`), and never imports a sibling — enforced by an AST test, not by convention.

THE FOUR AGENTS

Each agent owns its artifacts

AGENT	OWNS	ITS "PLUS"	MUST NOT
1 — Specification	specification.md, agents.md	Skill /write-spec	Write pipeline code — the spec is frozen first
2 — Code generation	pipeline/, frontend/, mcp/	MCP context7 , 3 documented queries	Write its own tests, or relax a spec rule
3 — Unit tests	tests/, the coverage gate	Hook blocking push under 80 %	Edit pipeline/ to make a test pass
4 — Documentation	README, HOWTORUN, deck	Author named in the README	Document behaviour it has not observed

agents.md is the contract all four work under: money rules, PII rules, and the rule that a failing test is never fixed by loosening the assertion.

DECISION RULES

Closed sets, not "e.g."

Validation — 10 checks, first failure wins

MISSING_FIELD · INVALID_TIMESTAMP ·
INVALID_ACCOUNT_FORMAT · SAME_ACCOUNT ·
UNSUPPORTED_CURRENCY · MALFORMED_AMOUNT ·
NON_POSITIVE_AMOUNT · AMOUNT_SCALE_MISMATCH ·
AMOUNT_OUT_OF_RANGE · UNSUPPORTED_TRANSACTION_TYPE

Scale follows the ISO 4217 exponent — 2 for USD, **0 for JPY**. "1000.5"
JPY is malformed input, not a value to round.

Fraud — additive, capped at 100

HIGH_VALUE $\geq$ 10 000	40
VERY_HIGH_VALUE $\geq$ 50 000	20
STRUCTURING [9 000, 10 000)	25
UNUSUAL_HOUR 00:00–04:59 UTC	20
CROSS_BORDER	15
UNATTENDED_CHANNEL (api)	10

LOW 0–24 · MEDIUM 25–59 · **HIGH $\geq$ 60 → held**. A score never travels
without the signals that produced it.

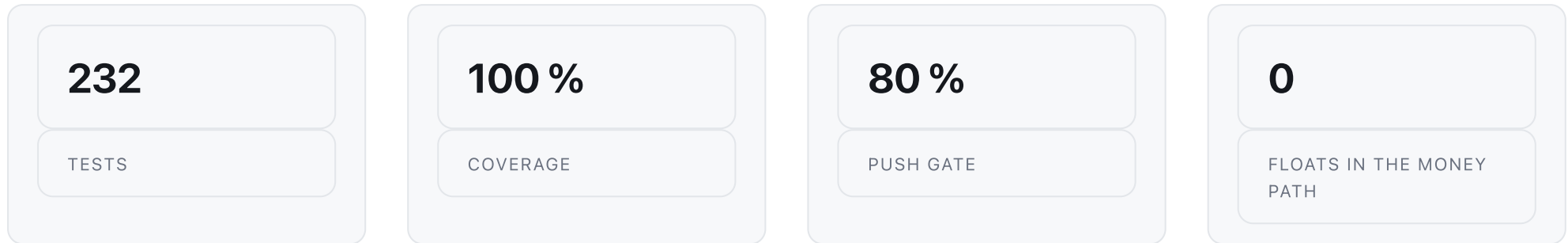
DEMO

Eight transactions, eight defined outcomes

TXN	AMOUNT	DECIDED BY	OUTCOME	WHY
TXN001	1 500.00 USD	settlement	settled	risk 0
TXN002	25 000.00 USD	settlement	settled	risk 40 HIGH_VALUE · CTR filed, did not block
TXN003	9 999.99 USD	compliance	held	WATCHLIST_MATCH · risk 25 STRUCTURING
TXN004	500.00 EUR	settlement	settled	risk 45 · 02:47 UTC, cross-border, api channel
TXN005	75 000.00 USD	fraud_detector	held	risk 60 HIGH → never reached settlement
TXN006	200.00 XYZ	validator	rejected	UNSUPPORTED_CURRENCY
TXN007	−100.00 GBP	validator	rejected	NON_POSITIVE_AMOUNT — not abs () -ed
TXN008	3 200.00 USD	settlement	settled	risk 0

4 settled · 2 held · 2 rejected — settled gross 30 242.50 USD, fees 38.11, net 30 204.39. This table is an acceptance fixture: the integration test asserts it row by row.

Tests that assert the absence of things



- **AST scan** — no `float()` call, no float literal in `pipeline/`.
- **AST scan** — no stage module imports a sibling stage.
- **After a full run** — no ACC-NNNN, no description anywhere in the audit log.
- `log_event` **raises** rather than writing a line carrying either.

The gate, verified by breaking it

160 untested functions → coverage **63.64 %** → hook exits **2**, push blocked.

One sabotaged assertion → suite red → exits **2**.

`ls -la` → exits **0**, suite never runs.

The threshold is passed to pytest by the gate, not read from `pytest.ini` — a gate you can lower by editing the file it reads is not a gate.

Two servers: one consumed, one built

context7 — consumed during code generation

Three queries, documented in `research-notes.md` with library IDs and what each one changed.

The useful one corrected code I had already written. POST `/api/run` used Starlette's `run_in_threadpool` from an `async def` — hand-rolling what FastAPI does from the signature. The docs' idiom is a plain `def` path operation. One dependency dropped out of the file.

pipeline-status — built with FastMCP

```
get_transaction_status(id) · list_pipeline_results() ·  
resource pipeline://summary
```

Read-only and **stateless**: stdio spawns a fresh process per session, so every call reads `shared/results/` from disk and can never serve a stale answer after a new run.

Returns the same projection the dashboard uses — account numbers and descriptions never leave the server.

What the process actually taught

- **The spec paid for itself at the §9 table.** Assigning every one of the 8 sample records a defined outcome *before* coding turned the integration test into a transcription job — and caught that a "compliance" stage with nothing to decide is untestable theatre.
- **Freeze the spec, then amend it deliberately.** Two implementation decisions drifted from the document; both were written back into it in the same commit. Silent drift is how a spec stops being trustworthy.
- **Verify by breaking, not by reading.** The coverage gate only became evidence once it actually blocked a push twice.
- **Documentation lookup beats recall.** context7 found a real mistake in code that already worked — the kind that passes tests and reviews.
- **Tests for absence are the ones that hold.** "No float in the money path" survives a refactor by a developer who never read the spec; a comment does not.
- **Two debugging traps worth remembering:** an MCP stdio client that pipes every request at once gets cut off at EOF; and a same-length edit with an older mtime leaves a stale .pyc that fails a test you already fixed.